

Итеративный состязательный атакователь ML-модели

Защита через состязательное обучение (adversarial training)



Проблема: состязательные примеры

Глубокие нейросети уязвимы: крошечное, незаметное глазу возмущение входа заставляет модель ошибаться с высокой уверенностью.

1

Незаметно

Возмущение $\|\delta\|_\infty \leq \epsilon$ визуально неотличимо от оригинала.

2

Опасно

Угроза для распознавания знаков, биометрии, медицины.

3

Универсально

Уязвимы практически все глубокие модели.

Цель и задачи

Цель

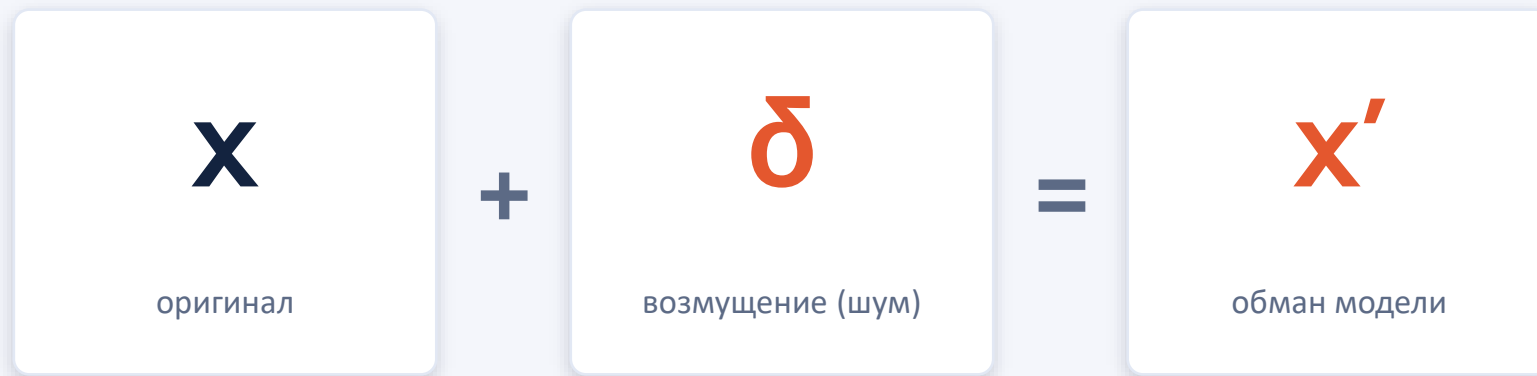
Разработать итеративный «атакователь» ML-модели и показать эффективность состязательного обучения как защиты за 10 итераций цикла «атака — защита».

Задачи

1. обучить базовую CNN на MNIST;
2. реализовать атакующий на ART (FGSM, PGD);
3. реализовать защиту — состязательное обучение;
4. оркестрировать цикл на 10 итераций с метриками;
5. визуализировать результаты;
6. экспортировать устойчивую модель в ONNX.

Что такое состязательный пример

Состязательный пример — вход $x' = x + \delta$, визуально неотличимый от исходного ($\|\delta\|_\infty \leq \epsilon$), но классифицируемый моделью неверно.



Идея (Goodfellow, 2014): сдвиг входа на ϵ вдоль знака градиента $\nabla_x J(\vartheta, x, y)$.

Атаки из ART: FGSM и PGD

FGSM — одношаговая

$$x' = x + \varepsilon \cdot \text{sign}(\nabla_x J)$$

- один шаг по градиенту;
- быстрая, но относительно слабая;
- база для понимания атак.

PGD — итеративная

$$x^{t+1} = \text{Proj}(x^t + \alpha \cdot \text{sign}(\nabla_x J))$$

- много шагов с проекцией на ε -окрестность;
- «сильнейшая атака первого порядка»;
- стандарт для оценки устойчивости.

Защита: состязательное обучение

Состязательное обучение (Madry, 2017) — дообучение модели на состязательных примерах. Минимаксная задача:

$$\min_{\theta} \mathbb{E}_{(x,y)} [\max_{\|\delta\|_{\infty} \leq \epsilon} \mathcal{J}(\theta, x + \delta, y)]$$

Внутренний max

приближается атакой PGD — ищем худшее возмущение.

Внешний min

обычный шаг оптимизации — учим модель не поддаваться.

Ключевая деталь

примеры генерируются заново для каждого батча (см. слайд 12).

Данные и модель

Данные — MNIST

- 70 000 изображений рукописных цифр 28×28;
- градации серого, значения в [0, 1];
- 60 000 — обучение, 10 000 — тест;
- загрузка средствами ART (load_mnist).

Модель — CNN

Вход 1×28×28

Conv 3×3, 32 + ReLU

Conv 3×3, 64 + ReLU

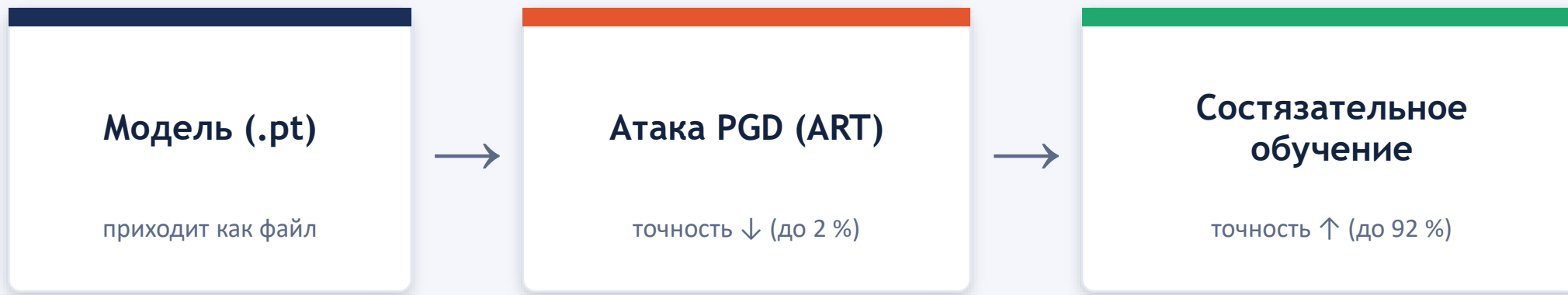
MaxPool 2×2 + Dropout

FC 128 + ReLU + Dropout

FC 10 (логиты)

Базовая точность ≈ 98.8 %

Итеративный цикл «атака ↔ защита»



🔄 **Повторяется 10 итераций.** Оценка — фиксированной атакой PGD. Устойчивость растёт **2 % → 92 %**.

Форматы модели: PyTorch .pt (pickle) на входе · экспорт устойчивой модели в ONNX.

Результаты: динамика по итерациям

98.8 %

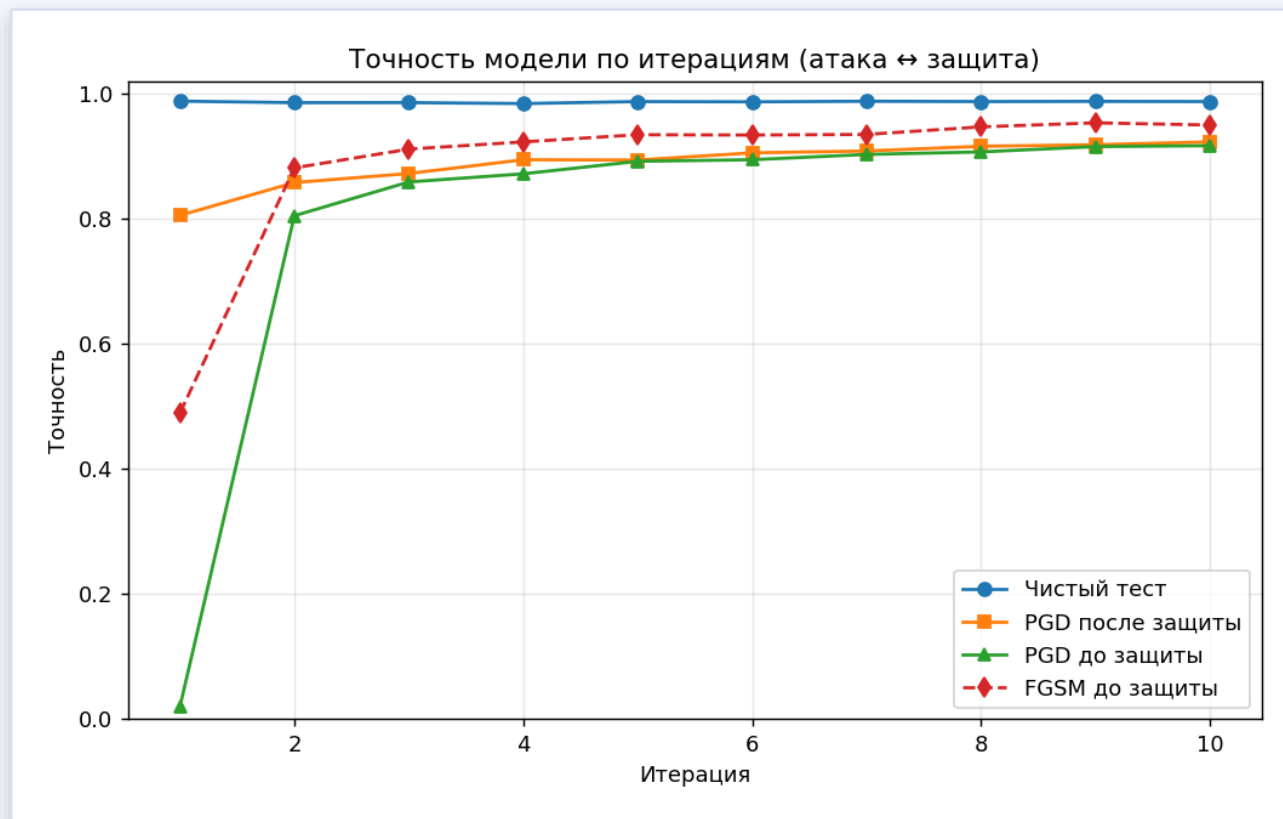
точность на чистом тесте (стабильна)

2.0 %

точность под PGD у базовой модели

92.3 %

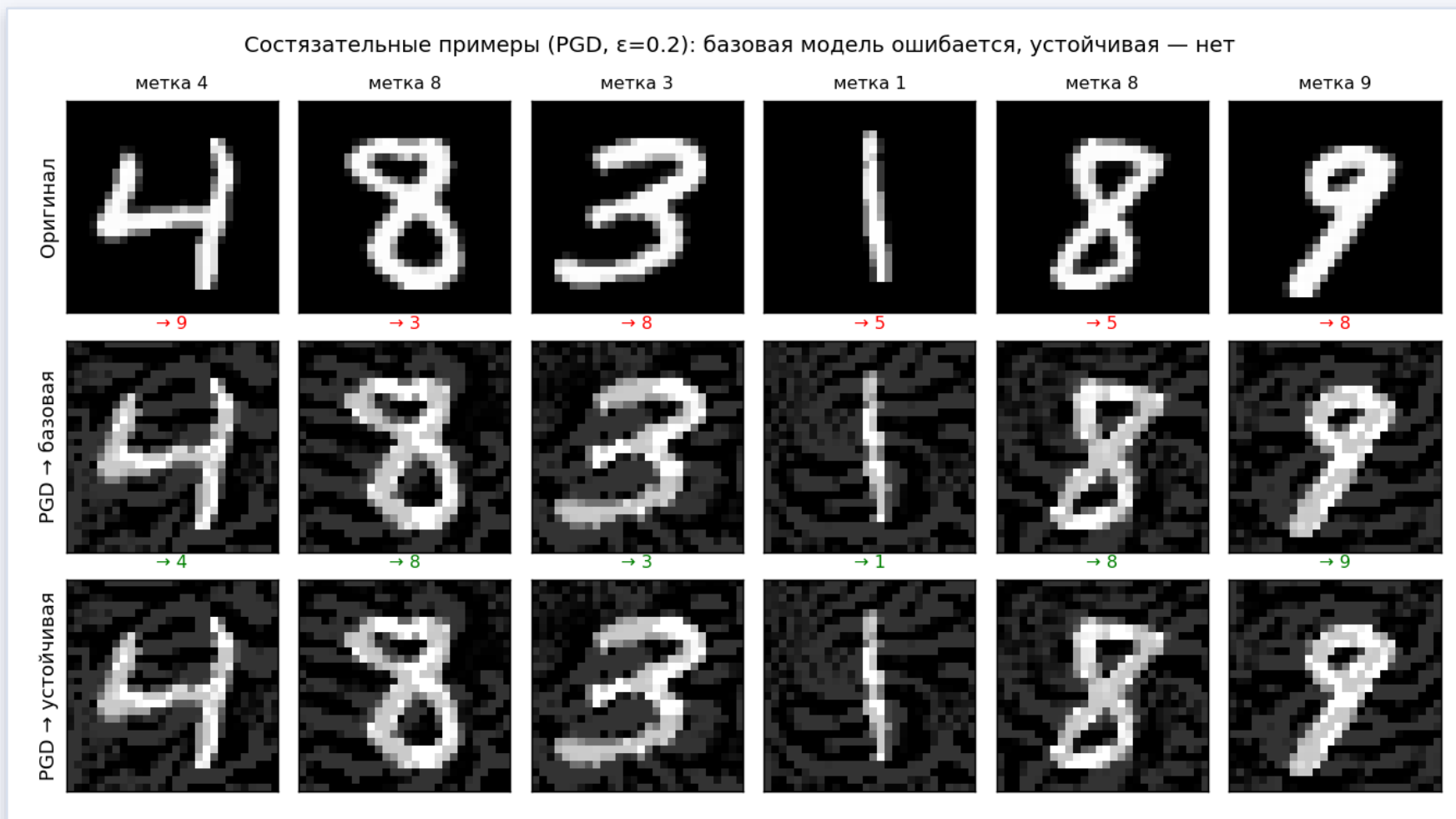
точность под PGD после 10 итераций защиты



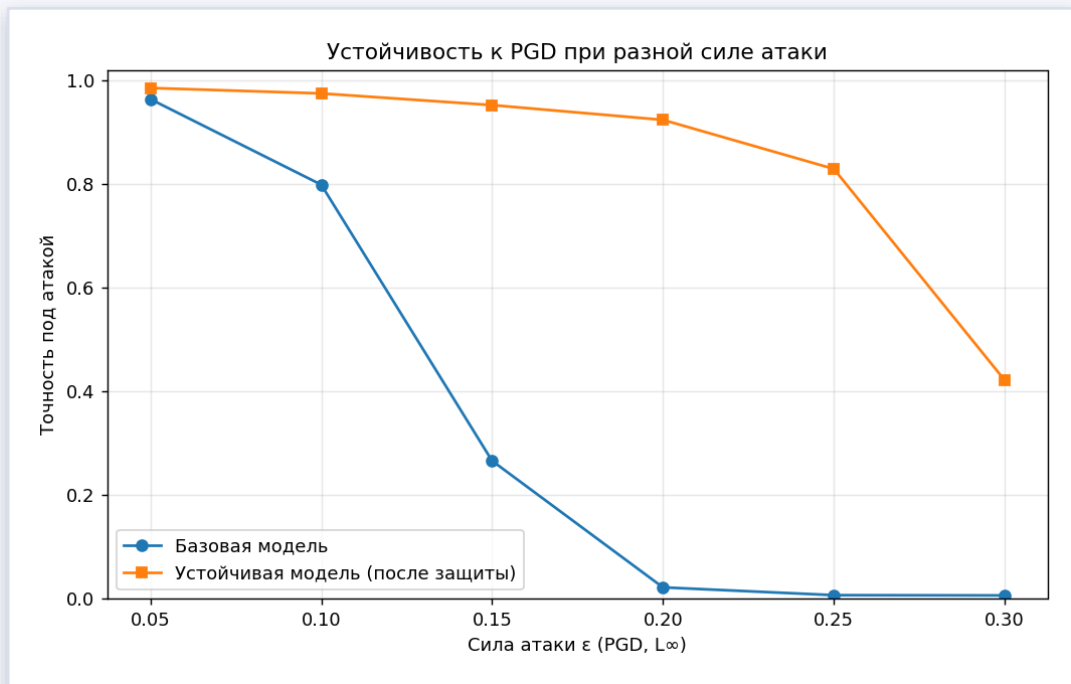
Точность по итерациям: атака ↔ защита

Результаты: одно возмущение — два исхода

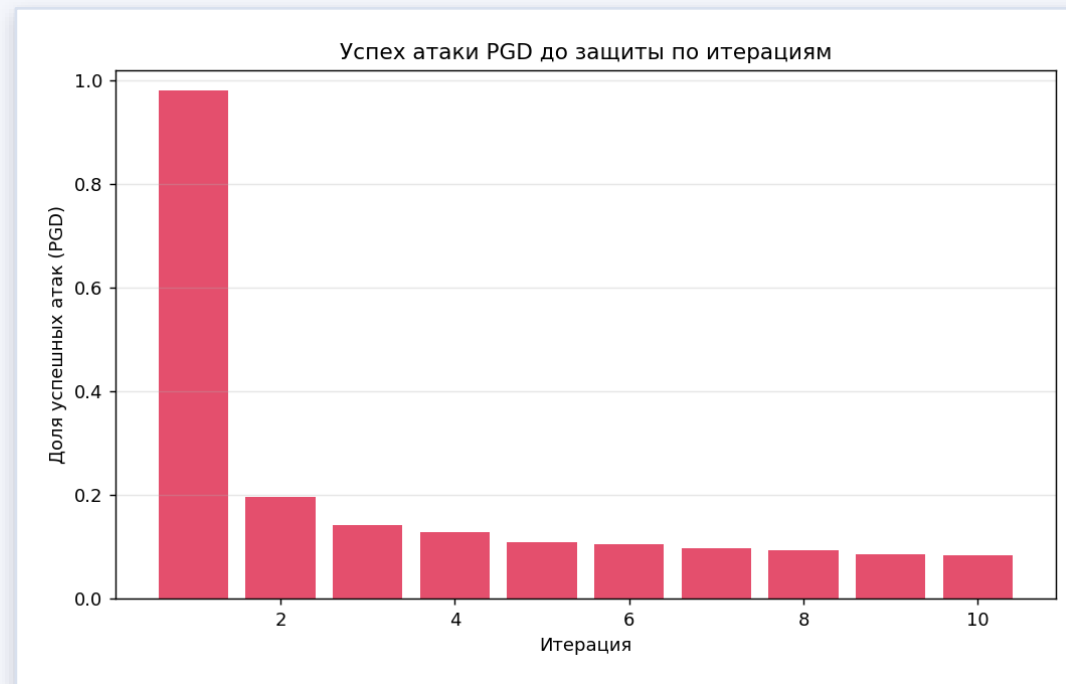
Одно и то же PGD-возмущение обманывает базовую модель (красное), но не устойчивую (зелёное).



Результаты: устойчивость и успех атаки



Устойчивость от силы атаки ϵ



Успех атаки PGD по итерациям

Устойчивая модель превосходит базовую при всех ϵ ; успех атаки PGD упал с **98 % до 8 %**.

Важная находка: адаптивность атаки

Качество защиты определяется не фактом обучения «на adversarial-данных», а адаптивностью атаки при обучении.

✗ Генерация 1× за итерацию

- атака строится по фиксированной подвыборке;
- модель «запоминает» возмущения;
- робастность не растёт ($\approx 1\text{--}12\%$).

✓ Генерация per-batch

- атака строится заново для каждого батча;
- против текущих весов модели;
- робастность растёт: $2\% \rightarrow 80\% \rightarrow 92\%$.

Выводы

- атака PGD снижает точность базовой модели с 98.8 % до 2.0 % (успех 98 %);
- состязательное обучение за 10 итераций поднимает устойчивость к PGD до 92.3 %;
- точность на чистых данных при этом сохраняется (~98.7 %);
- устойчивая модель превосходит базовую при всех значениях ϵ ;
- итоговая модель экспортирована в ONNX (расхождение с PyTorch $\sim 10^{-5}$);
- ключ к работающей защите — адаптивная (per-batch) генерация примеров.

Спасибо за внимание!

Вопросы?

Итеративный состязательный атакователь ML-модели · ART · adversarial training